FUNCTIONS

Functions:

A group of statements into a single logical unit is called as function.

Functions are used to perform the task.

Function is not called automatically.

Function body is executed whenever we call the function.

Function can be called any number of times.

Advantages of Functions:

1) Modularity
2) Reusability

Functions are divided into 4 categories:

1) Functions with arguments and with return value.
2) Functions with arguments and without return value.
3) Functions without arguments and with return value.
4) Functions without arguments and without return value.

1) Functions with arguments and with return value:

Syntax:

```
def function_name(arg1, arg2, …):
        Statement1
        Statement2
        =========
        return Statement
```

Example:

```
def max(a, b):
```

```
    if a>b:
        return a
    else:
        return b
x=max(10, 20)
print(x)
```

2) Functions with arguments and without return value:

Syntax:

```
def function_name(arg1, arg2, …):
        Statement1
        Statement2
        =========
```

Example:

```
def max(a, b):
    if a>b:
        print(a)
    else:
        print(b)
max(10, 20)
```

3) Functions without arguments and with return value:

Syntax:

```
def function_name():
        Statement1
        Statement2
```

=========

return Statement

Example:

```
def max():
    a, b = 10, 20
    if a>b:
        return a
    else:
        return b
x=max()
print(x)
```

4) Functions without arguments and without return value:

Syntax:

```
def function_name():
        Statement1
        Statement2
```

Example:

```
def max():
    a, b = 10, 20
    if a>b:
        print(a)
    else:
        print(b)
max()
```

Function Components:

```
def add(a, b):
    c=a+b
    return c
x=add(10, 20)
print(x)
```

In the above example

def add(a, b): => Function Header

def is a keyword and it is used to define a function, a & b are called formal arguments or formal parameters.

return c => Return Statement

return is a keyword and it is used to return a value.

x=add(10, 20) => Function Call Statement

10 & 20 are called actual arguments or actual parameters.

Types of arguments:

1) Non Default Arguments
2) Default Arguments
3) Arbitrary Arguments
4) Keyword Arguments
5) Non Keyword Arguments

1) Non Default Arguments

The arguments that are declared without assigning values are called default arguments.

At the time of calling a function we must pass values to non default arguments.

Example:

```
def add(a, b):
```

```
        c=a+b
        print(c)
    add(10, 20) => Valid
    add() => Error
```

2) Default Arguments

The arguments that are declared by assigning values in a function header are called default arguments.

At the time of calling function passing values are optional to default arguments.

Example:

```
    def add(a=10, b=20):
        c=a+b
        print(c)
    add() => Valid
    add(30, 40) => Valid
```

3) Arbitrary Arguments

The arguments that are prefixed with * & ** are called arbitrary arguments.

If the argument prefixed with * is tuple type.

If the argument prefixed with ** is dict type.

It allows to pass 0 to any number of arguments to a function.

Example1:

```
    def add(*a):
        print(type(a))
        for i in a:
            print(i)
```

```
add() => Valid
add(10) => Valid
add(33, 45) => Valid
add(31, 13, 54) => Valid
```

Example2:

```
def display(**a):
    print(type(a))
    for i in a:
        print(i, "=>", a[i])
display(x=10, y=20, z=30)
```

4) Keyword Arguments

The arguments that are passed with assigning to variables are called keyword arguments.

Example:

```
def add(a, b):
    print(a+b)
add(a=10, b=2) => Valid
add(b=2, a=10) => Valid
```

5) Non Keyword Arguments

The arguments that are passed without assigning to variables are called non keyword arguments.

Example:

```
def add(a, b):
    print(a+b)
```

```
add(10, 20)
```

Recursive Function:

A function that calls itself again and again is called as recursive function.

Example:

```
def fact(n):
    if n==1:
      return 1
    else:
      return n*fact(n-1)
x=fact(5)
print(x)
```

By

Mr. Venatesh Mansani

Naresh i Technologies